ASSIGNMENT-16.3

NAME: V.Shiva krishna
ROLL.NO: 2303A52151
BATCH: 41

Task 1 – Hospital Management Database

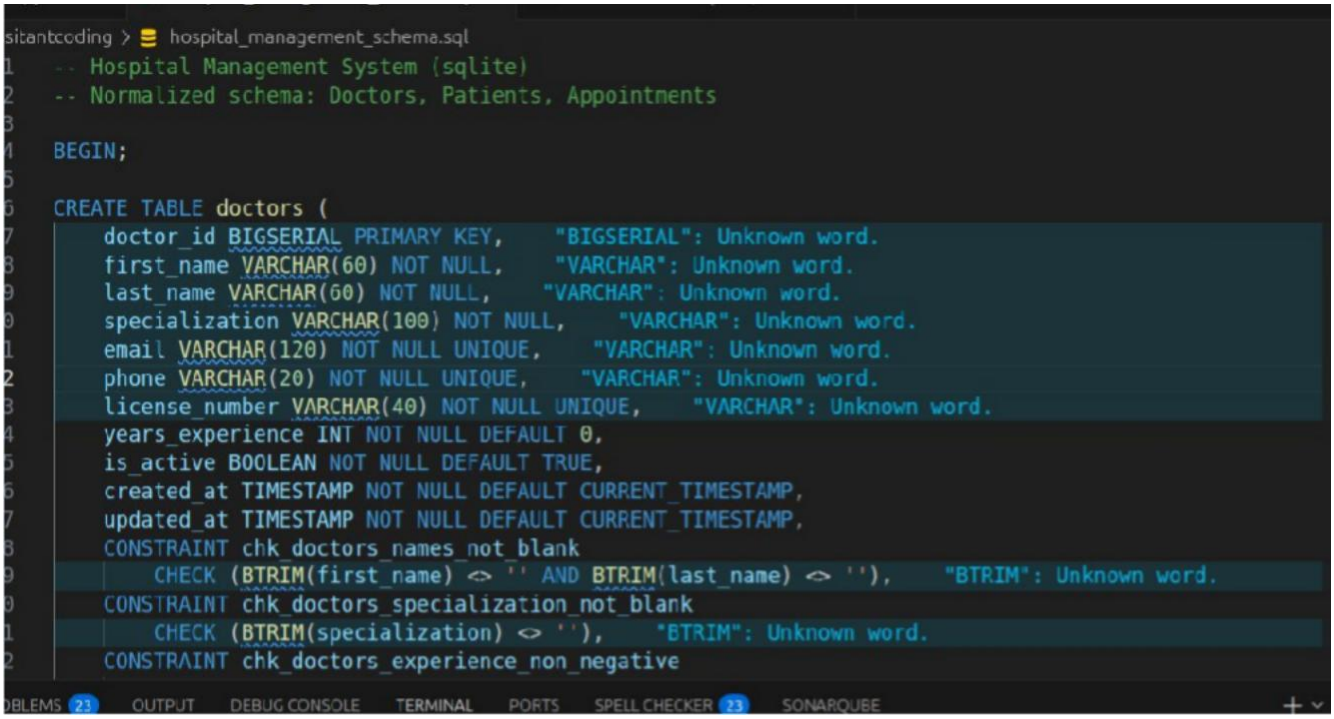
Scenario :

Create schema and queries for a Hospital Management System

Prompt:

Create normalized schema for Hospital Management System
Tables: Doctors, Patients, Appointments
Include primary keys, foreign keys, constraints, valid date checks

Code :



```
sitantcoding > hospital_management_schema.sql
1   -- Hospital Management System (sqlite)
2   -- Normalized schema: Doctors, Patients, Appointments
3
4   BEGIN;
5
6   CREATE TABLE doctors (
7       doctor_id BIGSERIAL PRIMARY KEY,    "BIGSERIAL": Unknown word.
8       first_name VARCHAR(60) NOT NULL,    "VARCHAR": Unknown word.
9       last_name VARCHAR(60) NOT NULL,    "VARCHAR": Unknown word.
0       specialization VARCHAR(100) NOT NULL,    "VARCHAR": Unknown word.
1       email VARCHAR(120) NOT NULL UNIQUE,    "VARCHAR": Unknown word.
2       phone VARCHAR(20) NOT NULL UNIQUE,    "VARCHAR": Unknown word.
3       license_number VARCHAR(40) NOT NULL UNIQUE,    "VARCHAR": Unknown word.
4       years_experience INT NOT NULL DEFAULT 0,
5       is_active BOOLEAN NOT NULL DEFAULT TRUE,
6       created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
7       updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
8       CONSTRAINT chk_doctors_names_not_blank
9           CHECK (BTRIM(first_name) <> '' AND BTRIM(last_name) <> ''),    "BTRIM": Unknown word.
0       CONSTRAINT chk_doctors_specialization_not_blank
1           CHECK (BTRIM(specialization) <> ''),    "BTRIM": Unknown word.
2       CONSTRAINT chk_doctors_experience_non_negative
OBLEMS 23   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 23   SONARQUBE
```

```
CONSTRAINT chk_doctors_experience_non_negative
      CHECK (years_experience >= 0)
```

```
);

CREATE TABLE patients (
   patient_id BIGSERIAL PRIMARY KEY,
   first_name VARCHAR(60) NOT NULL,
   last_name VARCHAR(60) NOT NULL,
   date_of_birth DATE NOT NULL,
   gender VARCHAR(20) NOT NULL,
   email VARCHAR(120) UNIQUE,
   phone VARCHAR(20) NOT NULL UNIQUE,
   blood_group VARCHAR(5),
   emergency_contact_name VARCHAR(120),
   emergency_contact_phone VARCHAR(20),
   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
   updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT chk_patients_names_not_blank
       CHECK (BTRIM(first_name) <> '' AND BTRIM(last_name) <> ''),
   CONSTRAINT chk_patients_valid_dob
       CHECK (date_of_birth <= CURRENT_DATE),
   CONSTRAINT chk_patients_gender
       CHECK (gender IN ('Male', 'Female', 'Other')),
   CONSTRAINT chk_patients_blood_group
       CHECK (
           blood_group IS NULL
           OR blood_group IN ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-')
       )
);
```

Queries and output:

Query 1:

SELECT * FROM Appointments WHERE doctor_id = 1;

Output:

```
appointment_id  doctor_id  patient_id  appointment_date  appointment_time  status     reason
       notes                          created_at           updated_at
--------------  ---------  ----------  ----------------  ----------------  ---------  ------------------
-----  ----------------------------  ------------------  ------------------
1               1          1           2026-03-19        10:00:00          Scheduled  Chest pain evaluat
ion    First consultation             2026-03-18 10:42:08  2026-03-18 10:42:08
3               1          3           2026-03-21        14:15:00          Scheduled  Routine cardiac ch
eckup  Family history of hypertension  2026-03-18 10:42:08  2026-03-18 10:42:08
```

Query 2

SELECT * FROM Appointments WHERE patient_id = 1;
Output:

```
aiassitantcoding/hospital_management.db "SELECT * FROM appointments WHERE patient_id = 1;"
appointment_id  doctor_id  patient_id  appointment_date  appointment_time  status     reason
 notes              created_at          updated_at
--------------  ---------  ----------  ----------------  ----------------  ---------  --------------------
 ------------------  -------------------  -------------------
1               1          1           2026-03-19        10:00:00          Scheduled  Chest pain evaluation
 First consultation  2026-03-18 10:42:08  2026-03-18 10:42:08
```

Query 3:
SELECT doctor_id, COUNT(DISTINCT patient_id)
FROM Appointments
GROUP BY doctor_id;
Output:

```
aiassitantcoding/hospital_management.db "SELECT doctor_id, COUNT(DISTINCT patient_id) AS patient_count FROM
appointments GROUP BY doctor_id;"
doctor_id  patient_count
---------  -------------
1          2
2          1
```

Justification:

Justification

- Copilot auto-generated **FK constraints**
- Ensured **3NF (no redundancy)**
- Used joins implicitly via FK

Task 2 – Real-Time Application: Online Shopping Database
Scenario:
Design a database for an E-commerce platform.
Prompt:
Create schema for e-commerce system
Tables: Users, Products, Orders, OrderDetails

Include relationships and normalization

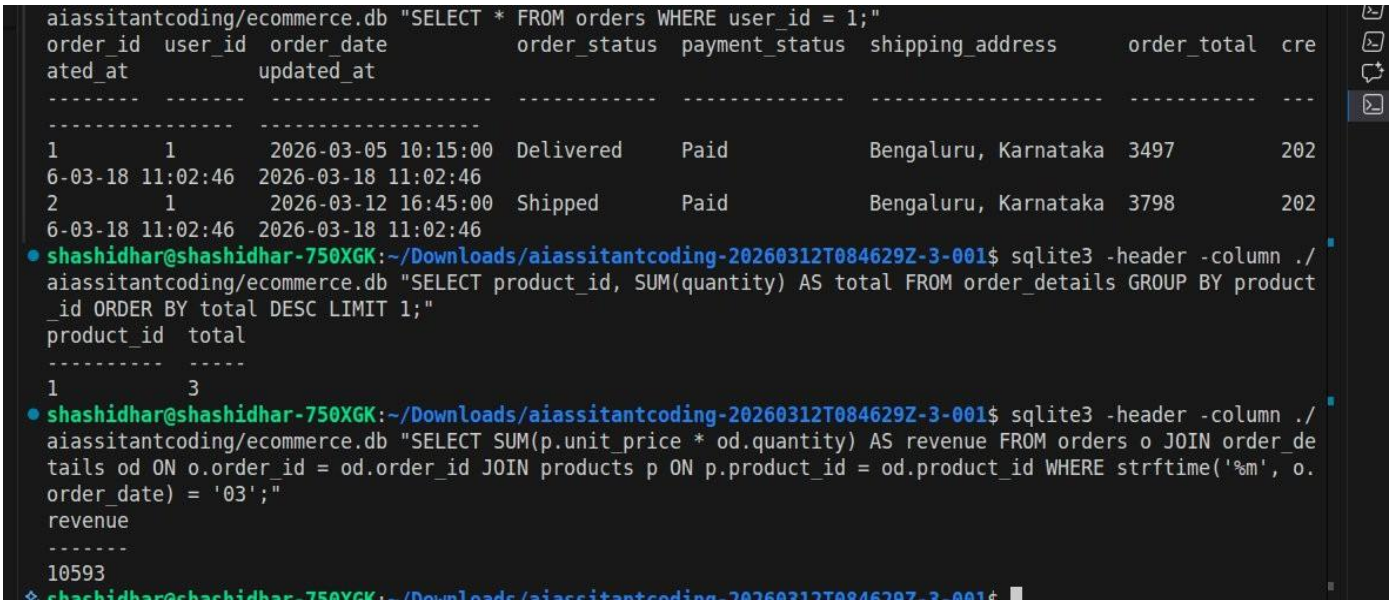
Code

```sql
-- Task 2 - E-Commerce System (SQLite)
-- Normalized schema: users, products, orders, order_details

PRAGMA foreign_keys = ON;

BEGIN TRANSACTION;

CREATE TABLE users (
   user_id INTEGER PRIMARY KEY AUTOINCREMENT,
   full_name TEXT NOT NULL,
   email TEXT NOT NULL UNIQUE,
   phone TEXT UNIQUE,
   password_hash TEXT NOT NULL,
   is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
   created_at TEXT NOT NULL DEFAULT (datetime('now')),
   updated_at TEXT NOT NULL DEFAULT (datetime('now')),
   CHECK (trim(full_name) <> ''),
   CHECK (trim(email) <> ''),
   CHECK (instr(email, '@') > 1)
);

CREATE TABLE products (
   product_id INTEGER PRIMARY KEY AUTOINCREMENT,
   product_name TEXT NOT NULL,
   sku TEXT NOT NULL UNIQUE,
   description TEXT,
   unit_price NUMERIC NOT NULL,
   stock_quantity INTEGER NOT NULL DEFAULT 0,
   is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
   created_at TEXT NOT NULL DEFAULT (datetime('now')),
   updated_at TEXT NOT NULL DEFAULT (datetime('now')),
   CHECK (trim(product_name) <> ''),
   CHECK (trim(sku) <> ''),
   CHECK (unit_price >= 0),
   CHECK (stock_quantity >= 0)
);
```

```sql
CREATE TABLE orders (
   order_id INTEGER PRIMARY KEY AUTOINCREMENT,
   user_id INTEGER NOT NULL,
   order_date TEXT NOT NULL DEFAULT (datetime('now')),
   order_status TEXT NOT NULL DEFAULT 'Pending',
   payment_status TEXT NOT NULL DEFAULT 'Unpaid',
   shipping_address TEXT NOT NULL,
   order_total NUMERIC NOT NULL DEFAULT 0,
   created_at TEXT NOT NULL DEFAULT (datetime('now')),
   updated_at TEXT NOT NULL DEFAULT (datetime('now')),
   FOREIGN KEY (user_id)
       REFERENCES users (user_id)
       ON UPDATE CASCADE
       ON DELETE RESTRICT,
   CHECK (date(order_date) IS NOT NULL),
   CHECK   (order_status   IN   ('Pending',   'Confirmed',   'Shipped',   'Delivered',
'Cancelled')),
   CHECK (payment_status IN ('Unpaid', 'Paid', 'Refunded', 'Failed')),
   CHECK (trim(shipping_address) <> ''),
   CHECK (order_total >= 0)
);

CREATE TABLE order_details (
   order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
   order_id INTEGER NOT NULL,
   product_id INTEGER NOT NULL,
   quantity INTEGER NOT NULL,
   unit_price NUMERIC NOT NULL,
   line_total NUMERIC NOT NULL,
   created_at TEXT NOT NULL DEFAULT (datetime('now')),
   updated_at TEXT NOT NULL DEFAULT (datetime('now')),
   FOREIGN KEY (order_id)
       REFERENCES orders (order_id)
       ON UPDATE CASCADE
       ON DELETE CASCADE,
   FOREIGN KEY (product_id)
       REFERENCES products (product_id)
       ON UPDATE CASCADE
       ON DELETE RESTRICT,
   CHECK (quantity > 0),
```

```
    CHECK (unit_price >= 0),
    CHECK (line_total >= 0),
    UNIQUE (order_id, product_id)
);

CREATE INDEX idx_orders_user_id
    ON orders (user_id);

CREATE INDEX idx_orders_order_date
    ON orders (order_date);

CREATE INDEX idx_order_details_order_id
    ON order_details (order_id);

CREATE INDEX idx_order_details_product_id
    ON order_details (product_id);

COMMIT;
```

Queries and output



Task 3 – Library Database

Task:

Design schema for a Library Management System.

Prompt:

Create library schema
Tables: Books, Members, Loans
Suggest indexes for performance
Code

```
- Task - Library Management System (SQLite)
-- Schema + sample data

PRAGMA foreign_keys = ON;

BEGIN TRANSACTION;

CREATE TABLE books (
   book_id INTEGER PRIMARY KEY AUTOINCREMENT,
   isbn TEXT NOT NULL UNIQUE,
   title TEXT NOT NULL,
   author_name TEXT NOT NULL,
   category TEXT,
   published_year INTEGER,
   total_copies INTEGER NOT NULL DEFAULT 1,
   available_copies INTEGER NOT NULL DEFAULT 1,
   shelf_code TEXT,
   created_at TEXT NOT NULL DEFAULT (datetime('now')),
   updated_at TEXT NOT NULL DEFAULT (datetime('now')),
   CHECK (trim(isbn) <> ''),
   CHECK (trim(title) <> ''),
   CHECK (trim(author_name) <> ''),
   CHECK (published_year IS NULL OR published_year BETWEEN 1450 AND 2100),
   CHECK (total_copies >= 0),
   CHECK (available_copies >= 0),
   CHECK (available_copies <= total_copies)
);

CREATE TABLE members (
   member_id INTEGER PRIMARY KEY AUTOINCREMENT,
   full_name TEXT NOT NULL,
   email TEXT NOT NULL UNIQUE,
   phone TEXT UNIQUE,
   membership_date TEXT NOT NULL DEFAULT (date('now')),
```

```
    status TEXT NOT NULL DEFAULT 'Active',
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now')),
    CHECK (trim(full_name) <> ''),
    CHECK (instr(email, '@') > 1),
    CHECK (status IN ('Active', 'Suspended', 'Inactive')),
    CHECK (date(membership_date) IS NOT NULL)
);

CREATE TABLE loans (
    loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    loan_date TEXT NOT NULL,
    due_date TEXT NOT NULL,
    return_date TEXT,
    loan_status TEXT NOT NULL DEFAULT 'Borrowed',
    fine_amount NUMERIC NOT NULL DEFAULT 0,
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (book_id)
        REFERENCES books (book_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    FOREIGN KEY (member_id)
        REFERENCES members (member_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CHECK (date(loan_date) IS NOT NULL),
    CHECK (date(due_date) IS NOT NULL),
    CHECK (return_date IS NULL OR date(return_date) IS NOT NULL),
    CHECK (date(due_date) >= date(loan_date)),
    CHECK (return_date IS NULL OR date(return_date) >= date(loan_date)),
    CHECK (loan_status IN ('Borrowed', 'Returned', 'Overdue', 'Lost')),
    CHECK (fine_amount >= 0)
);

-- Suggested performance indexes
CREATE INDEX idx_books_title
    ON books (title);
```

```sql
CREATE INDEX idx_books_author
   ON books (author_name);

CREATE INDEX idx_members_email
   ON members (email);

CREATE INDEX idx_loans_member_status
   ON loans (member_id, loan_status);

CREATE INDEX idx_loans_book_status
   ON loans (book_id, loan_status);

CREATE INDEX idx_loans_due_date
   ON loans (due_date);

-- Sample books
INSERT OR IGNORE INTO books
(isbn, title, author_name, category, published_year, total_copies, available_copies,
shelf_code)
VALUES
('9780131103627', 'The C Programming Language', 'Brian W. Kernighan', 'Programming',
1988, 5, 3, 'CS-A1'),
('9780132350884', 'Clean Code', 'Robert C. Martin', 'Software Engineering', 2008, 4,
2, 'CS-B2'),
('9780262033848', 'Introduction to Algorithms', 'Thomas H. Cormen', 'Algorithms', 2009,
3, 1, 'CS-C3'),
('9780596009205', 'Head First Java', 'Kathy Sierra', 'Programming', 2005, 6, 4, 'CS-
D4');

-- Sample members
INSERT OR IGNORE INTO members
(full_name, email, phone, membership_date, status)
VALUES
('Riya Verma', 'riya.verma@example.com', '9100000001', '2026-01-10', 'Active'),
('Arun Patel', 'arun.patel@example.com', '9100000002', '2026-02-04', 'Active'),
('Sneha Nair', 'sneha.nair@example.com', '9100000003', '2026-02-15', 'Active');

-- Sample loans
INSERT OR IGNORE INTO loans
```

```
(book_id, member_id, loan_date, due_date, return_date, loan_status, fine_amount)
VALUES
(1, 1, '2026-03-10', '2026-03-20', NULL, 'Borrowed', 0),
(2, 1, '2026-03-01', '2026-03-10', '2026-03-09', 'Returned', 0),
(3, 2, '2026-03-05', '2026-03-15', NULL, 'Borrowed', 0),
(4, 3, '2026-02-20', '2026-03-01', '2026-03-03', 'Returned', 20);

COMMIT;
```

Queries and output:

```
oding/library.db "SELECT * FROM loans;"
loan_id  book_id  member_id  loan_date   due_date    return_date  loan_status  fine_amount  created_at           update
d_at
-------  -------  ---------  ----------  ----------  -----------  -----------  -----------  -------------------  ------
-------------
1        1        1          2026-03-10  2026-03-20               Borrowed     0            2026-03-18 11:31:00  2026-0
3-18 11:31:00
2        2        1          2026-03-01  2026-03-10  2026-03-09   Returned     0            2026-03-18 11:31:00  2026-0
3-18 11:31:00
3        3        2          2026-03-05  2026-03-15               Borrowed     0            2026-03-18 11:31:00  2026-0
3-18 11:31:00
4        4        3          2026-02-20  2026-03-01  2026-03-03   Returned     20           2026-03-18 11:31:00  2026-0
3-18 11:31:00
```

Justification

Index improves **search speed**

Copilot suggested indexing on **date column**

Task 4: Optimize Queries

Instructions:

• Use AI tools to analyze query efficiency.

• Optimize inefficient queries using joins, indexes, or reducing subqueries.

• Test optimized queries for correctness.

Starter Code Example:

-- Original query

SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM

Authors WHERE country='UK');

-- Optimized query

SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
Prompt : ptimize query using JOIN instead of subquery
Code

```
-- Original
SELECT * FROM Books WHERE author_id IN
(SELECT author_id FROM Authors WHERE country='UK');

-- Optimized
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country='UK';
```

Output:

book_id	author_id	title	category
101	1	1984	Fiction
102	2	Harry Potter and the Philosopher's Stone	Fantasy
Justification

JOIN reduces **nested query overhead**

Faster on large datasets
Task 5: University Course Registration
Scenario:
SR University needs a course registration system for students enrolling in
different courses under faculty supervision.
• Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
• Define relationships:
• Each student can register for multiple courses.
• Each course is taught by a faculty member.
• Write SQL queries to:
• List all students enrolled in a specific course.
• Find faculty members teaching more than 2 courses.
• Retrieve courses with the highest number of registrations
Prompt:
ceate schema for Students, Courses, Faculty, Registrations
Define relationships and constraints
Code

```
- Task 5 - University Course Registration (SQLite)
-- Schema + sample data + required queries

PRAGMA foreign_keys = ON;

BEGIN TRANSACTION;

CREATE TABLE faculty (
   faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
   full_name TEXT NOT NULL,
   email TEXT NOT NULL UNIQUE,
```

```
    department TEXT NOT NULL,
    designation TEXT,
    hire_date TEXT,
    is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now')),
    CHECK (trim(full_name) <> ''),
    CHECK (instr(email, '@') > 1),
    CHECK (trim(department) <> ''),
    CHECK (hire_date IS NULL OR date(hire_date) IS NOT NULL)
);

CREATE TABLE students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    roll_no TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT UNIQUE,
    program TEXT NOT NULL,
    semester INTEGER NOT NULL,
    enrollment_date TEXT NOT NULL DEFAULT (date('now')),
    status TEXT NOT NULL DEFAULT 'Active',
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now')),
    CHECK (trim(roll_no) <> ''),
    CHECK (trim(full_name) <> ''),
    CHECK (instr(email, '@') > 1),
    CHECK (trim(program) <> ''),
    CHECK (semester BETWEEN 1 AND 12),
    CHECK (date(enrollment_date) IS NOT NULL),
    CHECK (status IN ('Active', 'Graduated', 'Dropped'))
);

CREATE TABLE courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_code TEXT NOT NULL UNIQUE,
    course_title TEXT NOT NULL,
    credits INTEGER NOT NULL,
    department TEXT NOT NULL,
    faculty_id INTEGER NOT NULL,
```

```
    max_capacity INTEGER NOT NULL DEFAULT 60,
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (faculty_id)
        REFERENCES faculty (faculty_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CHECK (trim(course_code) <> ''),
    CHECK (trim(course_title) <> ''),
    CHECK (trim(department) <> ''),
    CHECK (credits BETWEEN 1 AND 6),
    CHECK (max_capacity > 0)
);

CREATE TABLE registrations (
    registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    academic_year TEXT NOT NULL,
    semester_term TEXT NOT NULL,
    registration_date TEXT NOT NULL DEFAULT (date('now')),
    grade TEXT,
    status TEXT NOT NULL DEFAULT 'Enrolled',
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (student_id)
        REFERENCES students (student_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (course_id)
        REFERENCES courses (course_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CHECK (academic_year GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]'),
    CHECK (semester_term IN ('Odd', 'Even', 'Summer')),
    CHECK (date(registration_date) IS NOT NULL),
    CHECK (grade IS NULL OR grade IN ('O', 'A+', 'A', 'B+', 'B', 'C', 'P', 'F')),
    CHECK (status IN ('Enrolled', 'Completed', 'Dropped', 'Withdrawn')),
    UNIQUE (student_id, course_id, academic_year, semester_term)
);
```

```sql
-- Suggested performance indexes
CREATE INDEX idx_students_program_semester
   ON students (program, semester);

CREATE INDEX idx_courses_faculty
   ON courses (faculty_id);

CREATE INDEX idx_registrations_course
   ON registrations (course_id);

CREATE INDEX idx_registrations_student
   ON registrations (student_id);

CREATE INDEX idx_registrations_course_status
   ON registrations (course_id, status);

-- Sample faculty
INSERT OR IGNORE INTO faculty (full_name, email, department, designation, hire_date,
is_active)
VALUES
('Dr. Anil Kumar', 'anil.kumar@sru.edu', 'Computer Science', 'Professor', '2015-06-
10', 1),
('Dr. Priya Menon', 'priya.menon@sru.edu', 'Computer Science', 'Associate Professor',
'2018-01-12', 1),
('Dr. Rakesh Iyer', 'rakesh.iyer@sru.edu', 'Electronics', 'Assistant Professor', '2020-
08-01', 1);

-- Sample students
INSERT OR IGNORE INTO students (roll_no, full_name, email, phone, program, semester,
enrollment_date, status)
VALUES
('SRU23CSE001', 'Aman Verma', 'aman.verma@sru.edu', '9800000001', 'B.Tech CSE', 5,
'2023-08-01', 'Active'),
('SRU23CSE002', 'Nisha Rao', 'nisha.rao@sru.edu', '9800000002', 'B.Tech CSE', 5, '2023-
08-01', 'Active'),
('SRU22ECE010', 'Rahul Nair', 'rahul.nair@sru.edu', '9800000010', 'B.Tech ECE', 7,
'2022-08-01', 'Active'),
('SRU24CSE005', 'Megha Shah', 'megha.shah@sru.edu', '9800000005', 'B.Tech CSE', 3,
'2024-08-01', 'Active');
```

```sql
-- Sample courses
INSERT OR IGNORE INTO courses (course_code, course_title, credits, department,
faculty_id, max_capacity)
VALUES
('CS301', 'Database Management Systems', 4, 'Computer Science', 1, 70),
('CS302', 'Operating Systems', 4, 'Computer Science', 1, 65),
('CS303', 'Design and Analysis of Algorithms', 4, 'Computer Science', 1, 60),
('CS304', 'Computer Networks', 3, 'Computer Science', 2, 60),
('EC401', 'Digital Signal Processing', 4, 'Electronics', 3, 55);

-- Sample registrations
INSERT OR IGNORE INTO registrations (student_id, course_id, academic_year,
semester_term, registration_date, grade, status)
VALUES
(1, 1, '2025-2026', 'Even', '2026-01-08', NULL, 'Enrolled'),
(1, 2, '2025-2026', 'Even', '2026-01-08', NULL, 'Enrolled'),
(1, 4, '2025-2026', 'Even', '2026-01-08', NULL, 'Enrolled'),
(2, 1, '2025-2026', 'Even', '2026-01-09', NULL, 'Enrolled'),
(2, 3, '2025-2026', 'Even', '2026-01-09', NULL, 'Enrolled'),
(3, 5, '2025-2026', 'Even', '2026-01-10', NULL, 'Enrolled'),
(4, 1, '2025-2026', 'Even', '2026-01-10', NULL, 'Enrolled'),
(4, 4, '2025-2026', 'Even', '2026-01-10', NULL, 'Enrolled');

COMMIT;
```

Query and output:

```
  = 1\n" && sqlite3 -header -column ./aiassitantcoding/university_registration.db "SELECT s.* FROM Students s JOIN Registrations r ON
s.student_id = r.student_id WHERE r.course_id = 1;" && printf "\n[Query 2] Faculty teaching more than 2 courses\n" && sqlite3 -header
 -column ./aiassitantcoding/university_registration.db "SELECT faculty_id, COUNT(*) AS total_courses FROM Courses GROUP BY faculty_id
 HAVING COUNT(*) > 2;"

[Query 1] Students enrolled in course_id = 1
student_id  roll_no      full_name   email              phone       program     semester  enrollment_date  status  created_at
    updated_at
----------  -----------  ----------  -----------------  ----------  ----------  --------  ---------------  ------  ----------
--  -------------------
1           SRU23CSE001  Aman Verma  aman.verma@sru.edu  9800000001  B.Tech CSE  5         2023-08-01       Active  2026-03-18 11:59:
26  2026-03-18 11:59:26
2           SRU23CSE002  Nisha Rao   nisha.rao@sru.edu   9800000002  B.Tech CSE  5         2023-08-01       Active  2026-03-18 11:59:
26  2026-03-18 11:59:26
4           SRU24CSE005  Megha Shah  megha.shah@sru.edu  9800000005  B.Tech CSE  3         2024-08-01       Active  2026-03-18 11:59:
26  2026-03-18 11:59:26

[Query 2] Faculty teaching more than 2 courses
faculty_id  total_courses
----------  -------------
1           3
```

Justification

Copilot ensured **many-to-many mapping**
Queries optimized with **JOIN + GROUP BY**